CS6650 Midterm Mastery

Steps I – III: From HW6 Results to Crash & Recovery

Weifan Li | Distributed Systems | February 2026

Go | ECS Fargate | Terraform | Locust | Circuit Breaker

Agenda

Step I: HW6 Results

Single instance bottleneck → horizontal scaling with ALB

Step II: Key Learnings

CAP, MapReduce, IaC, Parnas, and hands-on scaling

Step III: Crash Demo

Cascading failure with a slow downstream service

Step III: Recovery

Fail Fast + Circuit Breaker + Bulkhead patterns

Step I: Homework 6 Results

Go Product Search Service on ECS Fargate (0.25 vCPU, 512 MB)

Part II: Single Instance

- CPU scaled linearly with concurrency

Memory stayed flat (~11%)

→ CPU-bound workload confirmed

15% CPU	46% CPU
5 Users	50 Users

Part III: ALB + Auto Scaling

- 500 users for 5 minutes

Auto Scaled: 2 → 3 → 4 instances

Zero failures, stable response time

80.7%	~30s
CPU Peak	Recovery

Takeaway: Horizontal scaling with ALB is the correct long-term solution for stateless, compute-bound services.

Step II: What I Found Valuable

CAP & FLP

Partitions are inevitable. The real choice: consistency or availability?

MapReduce

Constrained abstractions enable automatic parallelization and fault tolerance.

Terraform + Docker

From 20 clicks to one command. IaC enables reproducibility at scale.

Parnas (1972)

Information hiding → microservice boundaries. 50-year-old ideas still relevant.

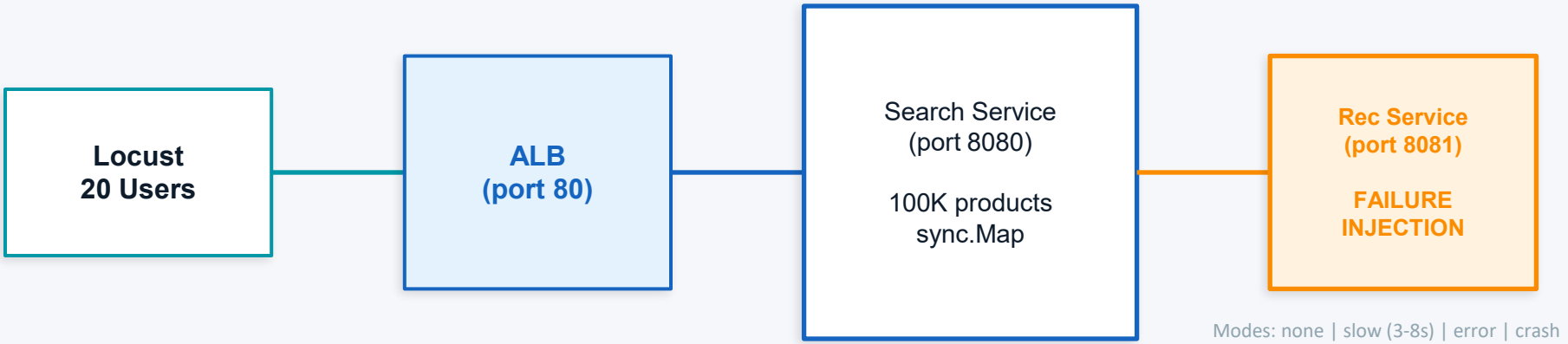
HW6 Scaling

Theory met practice: CPU bottleneck → ALB + Auto Scaling → problem solved.

Failure Models

8 Fallacies, halting/omission/Byzantine failures — vocabulary for real problems.

Step III: Crash & Recovery Architecture



RESILIENCE=false: waits forever, cascading failure

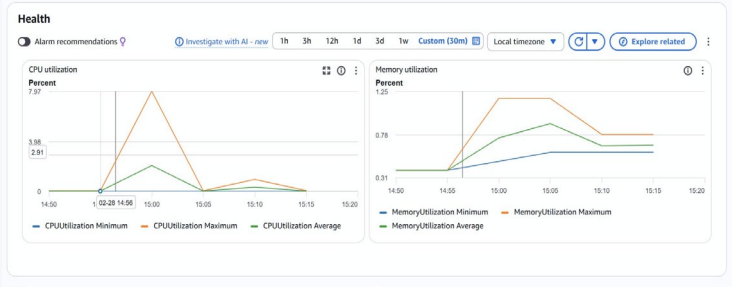
RESILIENCE=true: timeout + circuit breaker + bulkhead

Deployed via Terraform to AWS ECS Fargate | Service Discovery for inter-service communication

Phase 1: The Crash

No resilience — downstream goes slow (3-8s), everything dies

Metric	Baseline	Under Failure	Impact
RPS	89	4	–95%
Avg Latency	16 ms	4,644 ms	290x
Median	15 ms	5,200 ms	347x
Max	151 ms	8,015 ms	53x



CloudWatch: CPU < 8% — bottleneck is I/O, not CPU

Key Insight: The system didn't return errors — it returned successful responses, just 290x slower. Every goroutine blocked waiting for the slow downstream, creating a silent cascading failure.

```
(.venv) PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Midterm\Part II\crash-recovery> curl "http://crash-recovery-alb
-1591838157.us-west-2.elb.amazonaws.com/metrics"


StatusCode        : 200
StatusDescription : OK
Content           : {"failed_recs":0,"fallback_recs":0,"resilience_enabled":false,"successful_recs":6587,"timeout_recs":0,"
                    total_searches":6587}

RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Content-Length: 125
                    Content-Type: application/json
                    Date: Sat, 28 Feb 2026 23:16:51 GMT

                    {"failed_recs":0,"fallback_recs":0,"resilience_enabled":false,"succ...

Forms             : {}
Headers           : {[Connection, keep-alive], [Content-Length, 125], [Content-Type, application/json], [Date, Sat, 28 Feb
                    2026 23:16:51 GMT]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 125
```

resilience_enabled: false, all requests slow but "successful"

Phase 2: The Fix

Three resilience patterns from Sam Newman's Building Microservices

Fail Fast

500ms Timeout

Don't wait 3-8s for a slow service. Time out after 500ms and return fallback data.

Circuit Breaker

5 fails → OPEN

After 5 timeouts, stop calling downstream for 15s. Serve cached fallback instantly.

Bulkhead

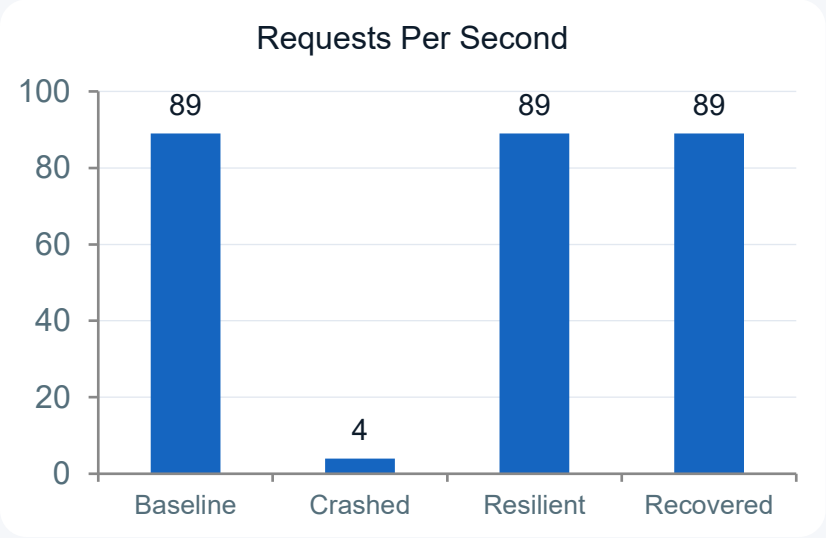
Max 10 concurrent

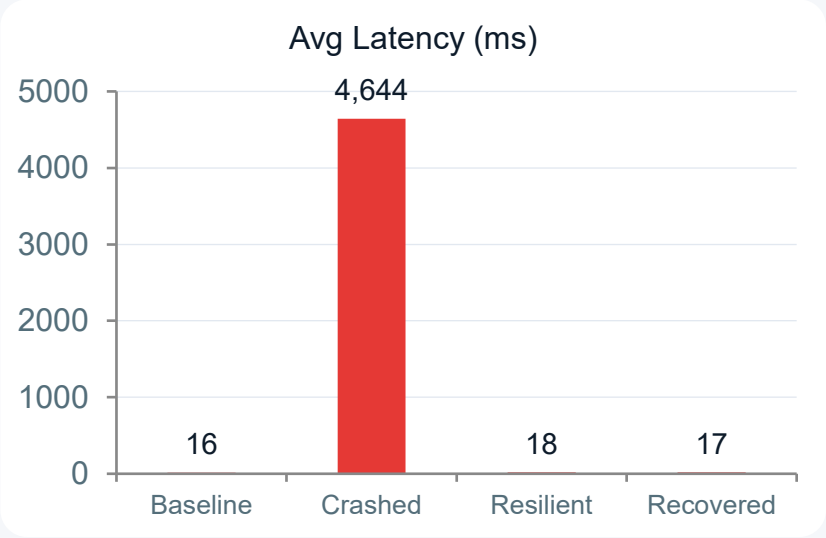
Only 10 goroutines can wait on downstream at once. The rest get fallback immediately.

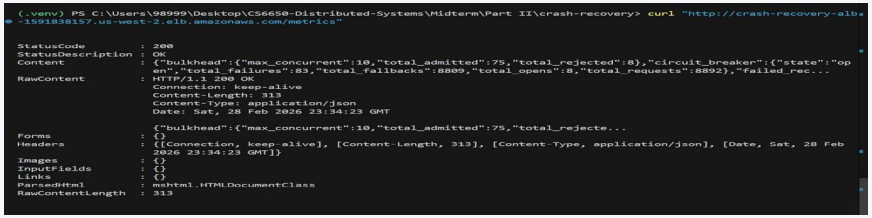
Result: Same failure conditions (downstream in slow mode), RPS recovered from **4 → 89**. Avg latency from **4,644ms → 18ms**. Zero failures.

One Terraform variable change: resilience_enabled = false → true, terraform apply

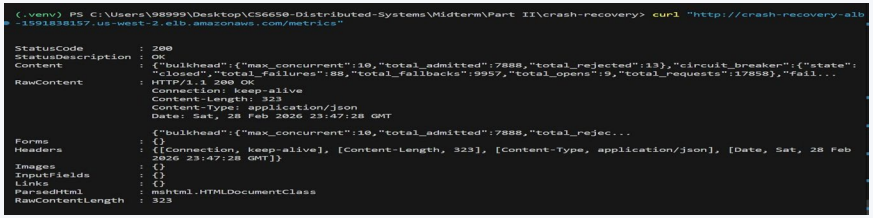
Full Results Comparison







Circuit breaker: OPEN, 8,809 fallbacks



Circuit breaker: CLOSED, fully recovered

Key Takeaways

01 Cascading failures are silent killers

No errors, just 290x slower. Monitor throughput, not just error rates.

02 CPU ≠ the bottleneck

CPU was 8% during the crash. I/O wait from blocked goroutines was the real problem.

03 Defense in depth works

Fail Fast + Circuit Breaker + Bulkhead = 100% throughput recovery under identical failure.

04 Degrade gracefully, don't fail hard

Fallback recommendations > 8-second timeouts. Users stay happy.

Thank You

Questions?

Weifan Li | CS6650 Distributed Systems | February 2026

Code: GitHub | Infra: Terraform + ECS Fargate | Testing: Locust